

Développement de l'Application "Car Dealer" (JavaFX & MySQL)

Matisse Kra

BTS SIO 1

Introduction et Contexte du Projet	2
1. Conception de la Base de Données et des Modèles	2
2. Création de l'Interface Graphique (Vues FXML)	4
3. Navigation et Architecture des Contrôleurs	6
4. Connexion à la Base de Données	8
5. Implémentation du CRUD	10
Conclusion et Suite du Projet	14

Introduction et Contexte du Projet

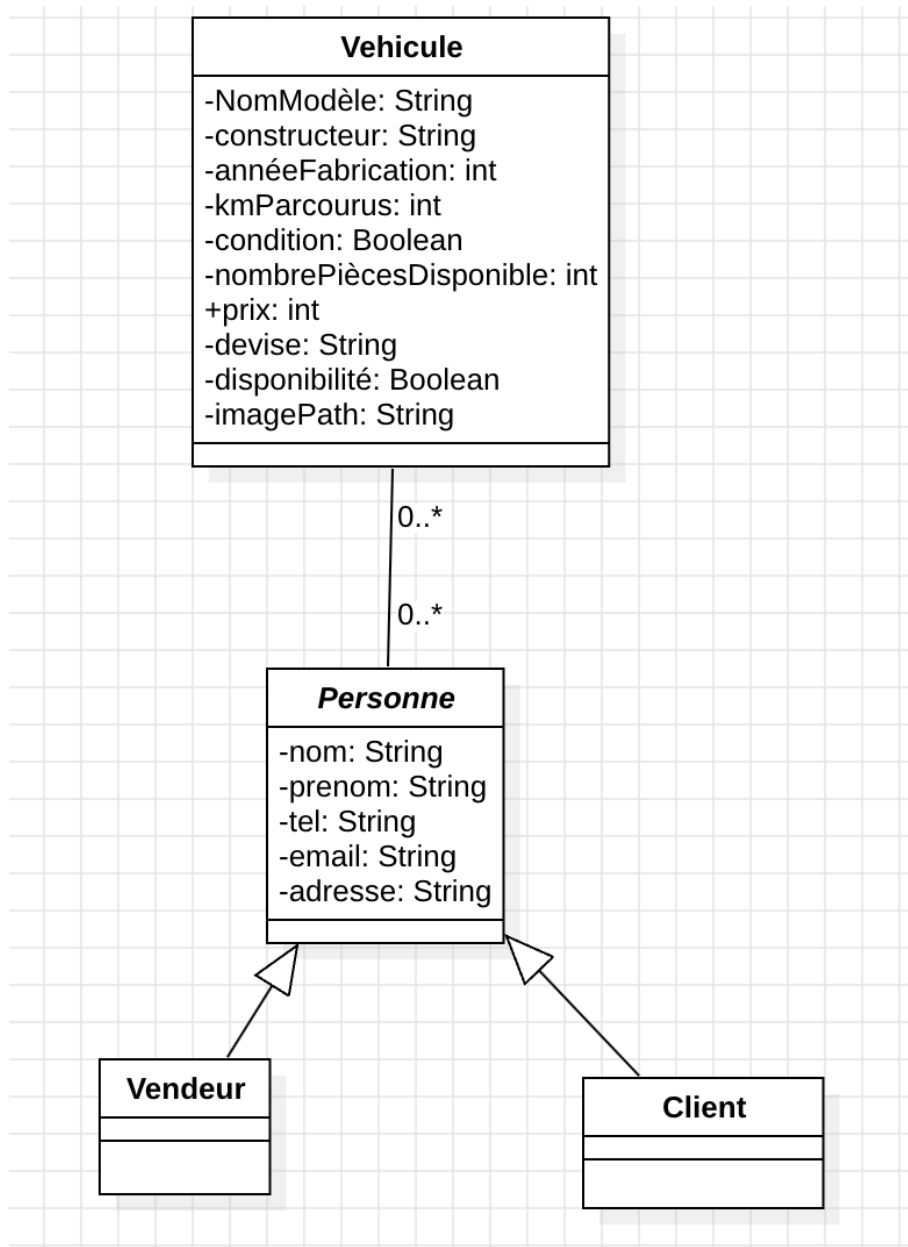
L'objectif de ce projet était de concevoir une application de gestion pour une concession automobile ("Car Dealer"). L'application doit permettre d'administrer les véhicules, les vendeurs et les clients via une interface graphique interactive. Ce TP a été réalisé en Java avec le framework JavaFX pour l'interface utilisateur, Maven pour la gestion des dépendances, et MySQL pour la persistance des données. L'architecture du projet respecte le modèle MVC (Modèle-Vue-Contrôleur), séparant clairement la logique métier, l'interface graphique et les actions de l'utilisateur.

1. Étape 1 : Conception de la Base de Données et des Modèles

Avant de coder l'interface, la fondation du projet a reposé sur la structuration des données.

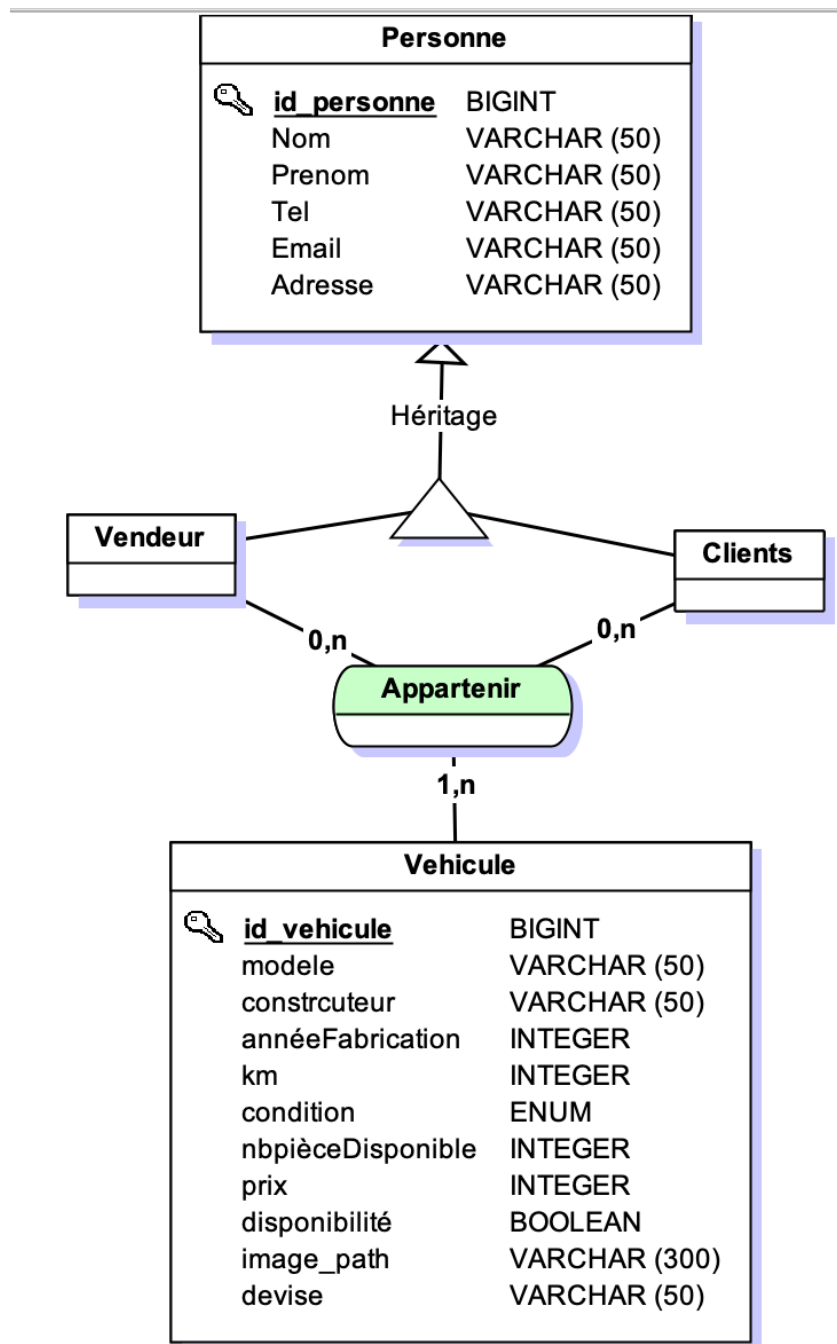
- **Création des Modèles (Classes Java)** : Modélisation des données : élaboration préalable d'un diagramme UML pour structurer les classes *Vehicule*, *Personne*, *Vendeur* et *Client*. Ces modèles incluent l'ensemble des attributs nécessaires, leurs constructeurs ainsi que les accesseurs (getters) et mutateurs (setters) correspondants.

diagramme uml



- **Base de données MySQL** : Création des tables correspondantes, avec une attention particulière à la table vehicle intégrant VARCHAR, INT, DOUBLE et LONGBLOB pour les images.

mcd



les tables sql

Table	Action	Lignes	Type	Interclassement	Taille	Perte
☐ Appartenir	Parcourir Structure Rechercher Insérer Vider Supprimer	0	InnoDB	utf8mb4_general_ci	48,0 kio	-
☐ Clients	Parcourir Structure Rechercher Insérer Vider Supprimer	0	InnoDB	utf8mb4_general_ci	16,0 kio	-
☐ Personne	Parcourir Structure Rechercher Insérer Vider Supprimer	0	InnoDB	utf8mb4_general_ci	16,0 kio	-
☐ vehicule	Parcourir Structure Rechercher Insérer Vider Supprimer	0	InnoDB	utf8mb4_general_ci	16,0 kio	-
☐ Vendeur	Parcourir Structure Rechercher Insérer Vider Supprimer	0	InnoDB	utf8mb4_general_ci	16,0 kio	-

La table vehicule

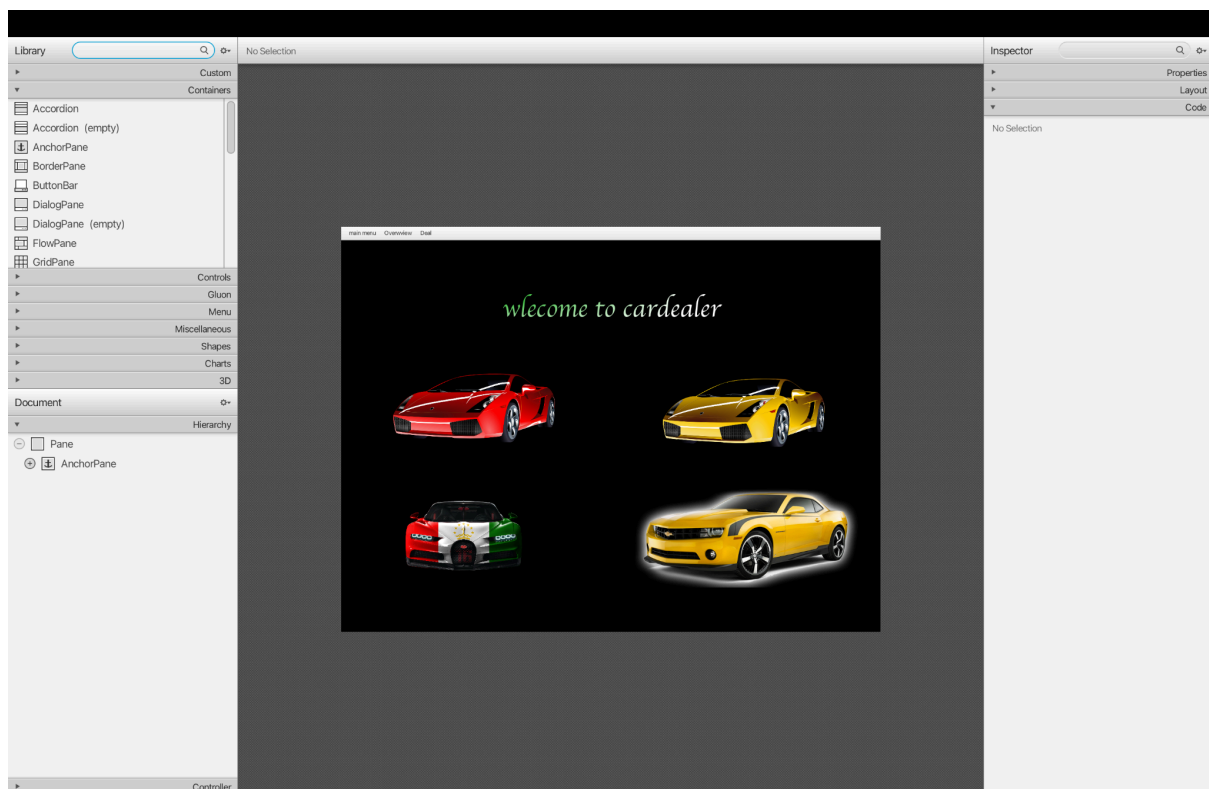
#	Nom	Type	Interclassement	Attributs	Null	Valeur par défaut	Commentaires	Extra	Action
☐	1 id	int(11)			Non	Aucun(e)		AUTO_INCREMENT	Modifier Supprimer Plus
☐	2 vehicle_name	varchar(100)	utf8mb4_general_ci		Oui	NULL			Modifier Supprimer Plus
☐	3 manufacturer	varchar(50)	utf8mb4_general_ci		Non	Aucun(e)			Modifier Supprimer Plus
☐	4 construction_year	int(11)			Non	Aucun(e)			Modifier Supprimer Plus
☐	5 km_stood	double			Oui	NULL			Modifier Supprimer Plus
☐	6 condition	varchar(50)	utf8mb4_general_ci		Oui	NULL			Modifier Supprimer Plus
☐	7 number_of_pieces	int(11)			Non	Aucun(e)			Modifier Supprimer Plus
☐	8 price	double			Non	Aucun(e)			Modifier Supprimer Plus
☐	9 availability	varchar(50)	utf8mb4_general_ci		Oui	NULL			Modifier Supprimer Plus
☐	10 image	longblob			Oui	NULL			Modifier Supprimer Plus
☐	11 currency	varchar(50)	utf8mb4_general_ci		Non	Aucun(e)			Modifier Supprimer Plus

2. Étape 2 : Création de l'Interface Graphique (Vues FXML)

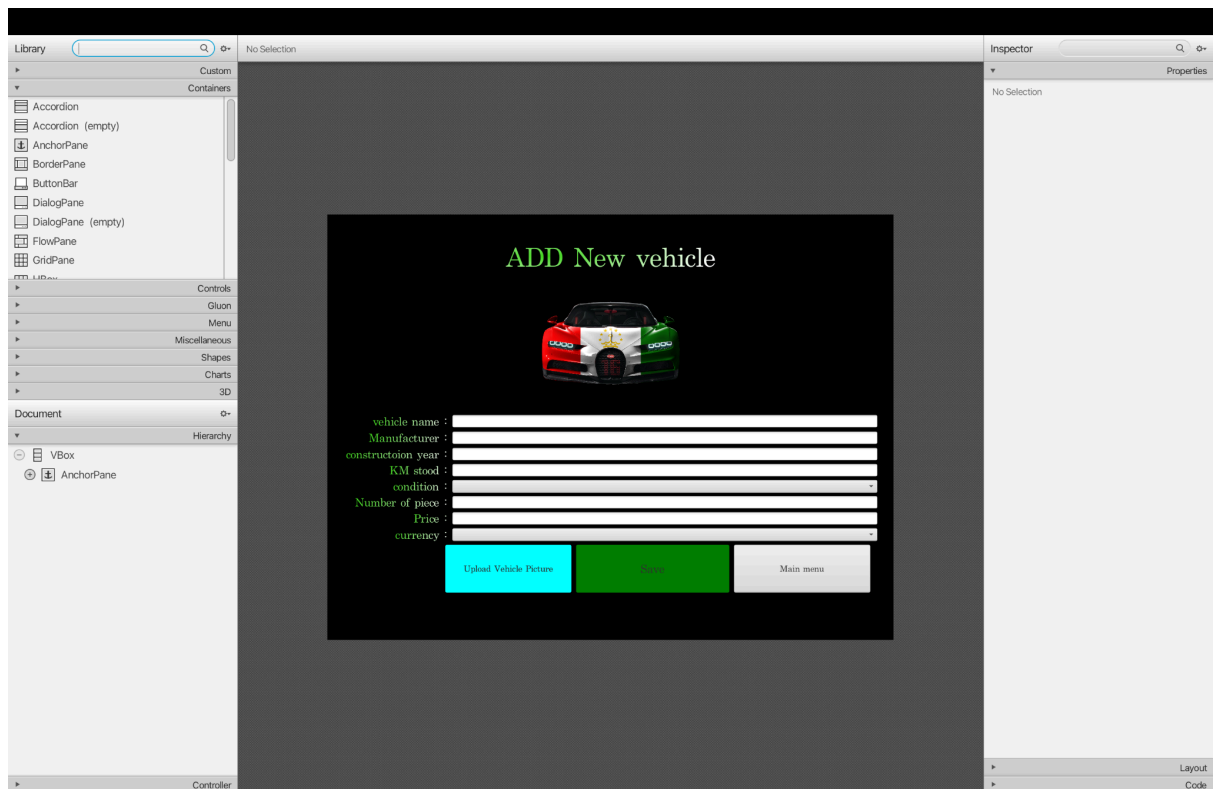
L'interface graphique a été conçue à l'aide de Scene Builder, générant des fichiers .fxml stockés dans le dossier resources.

- **Agencement (Layouts)** : Utilisation de conteneurs tels que AnchorPane, VBox, et GridPane pour structurer proprement les formulaires et les tableaux.
- **Composants graphiques** : Intégration de TextField, ComboBox paramétrés, Button, et ImageView pour l'affichage des photos.
- **Identifiants (fx:id)** : Chaque élément interactif a reçu un fx:id strict pour permettre au Contrôleur Java d'en prendre le contrôle.

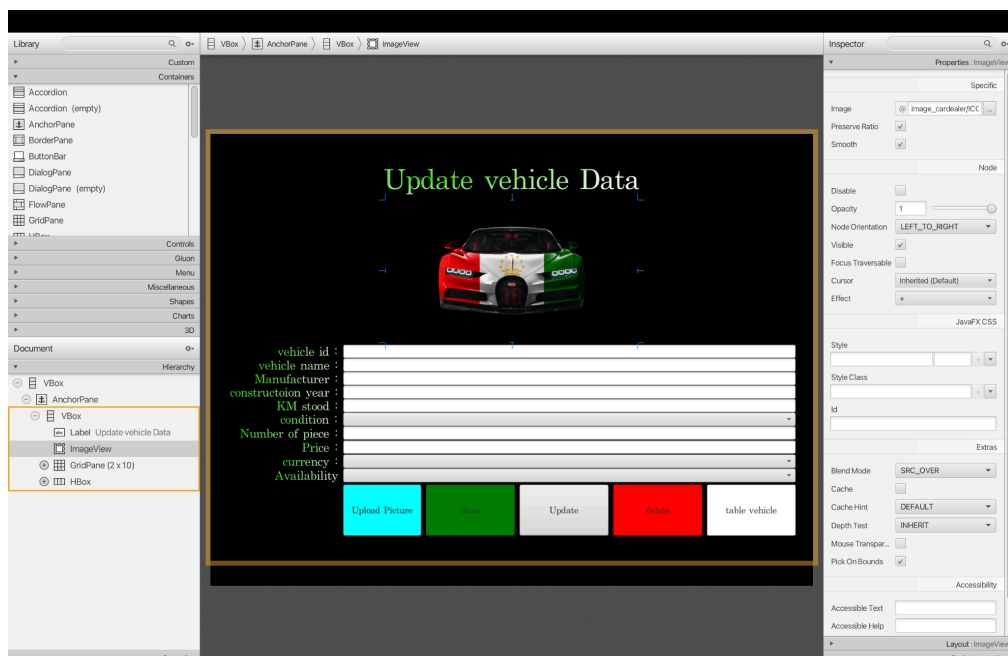
vue menu



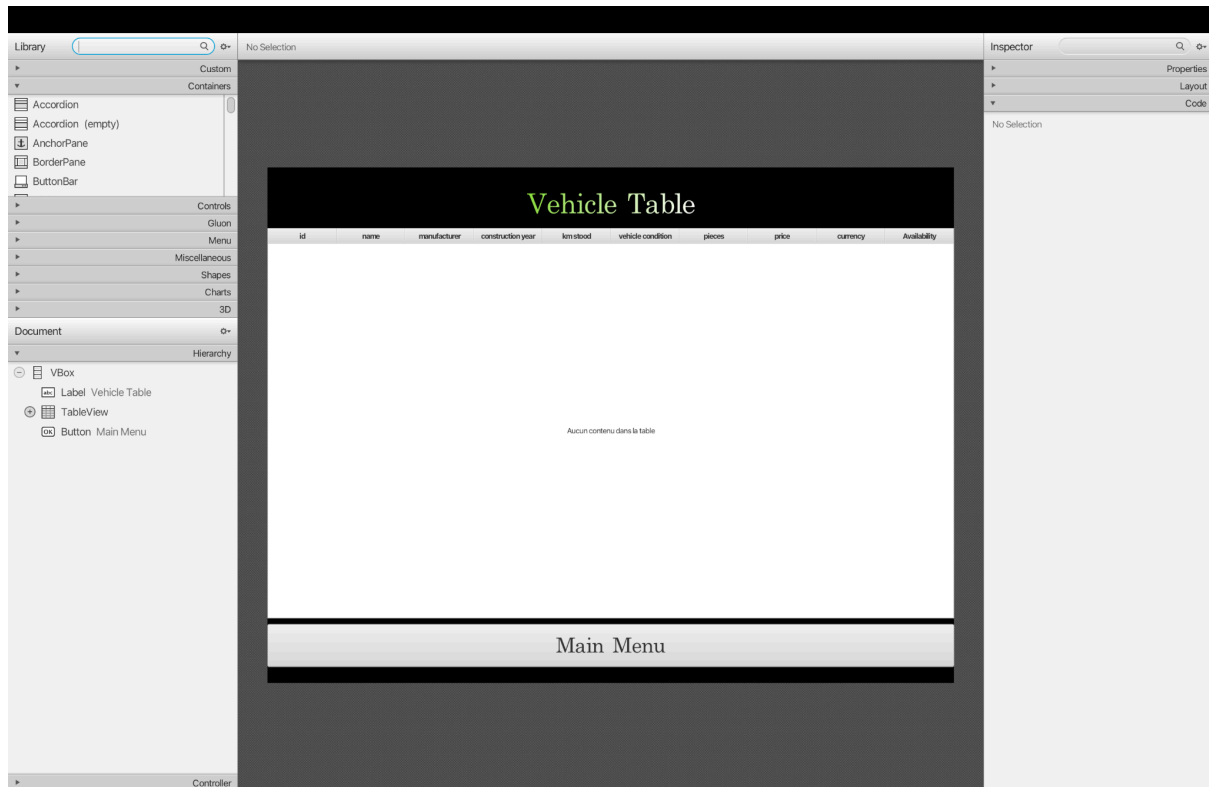
vue new vehicle



vue update véhicule



vue table vehicule

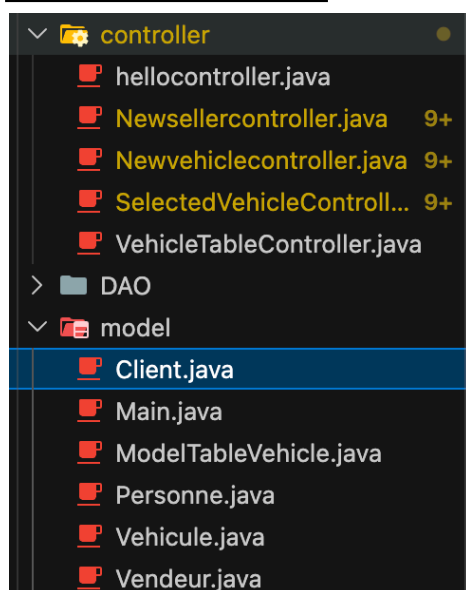


3. Étape 3 : Navigation et Architecture des Contrôleurs

Chaque vue FXML est pilotée par son propre fichier Contrôleur Java.

- **Liaison Vue/Contrôleur** : L'annotation `@FXML` a été utilisée pour lier les composants graphiques du FXML aux variables Java.

La vue des controller



L'annotation @FXML

```
@FXML public Button buttonuploadVehiclePicture;
@FXML public Button button_Save;      Rename this f
@FXML public Button Button_mainMenu;    Rename th

@FXML public TextField txtName;
@FXML public TextField txtManufacturer;
@FXML public TextField txt_constructoionYear;
@FXML public TextField txy_KMstood;
@FXML public ComboBox<String> txt_condition;
@FXML public ComboBox<String> txt_currency;
@FXML public TextField txt_Number_of_piece;
@FXML public TextField txt_price;
@FXML public ImageView imageviewaddnewvehicle;
```

- **Système de navigation** : Le passage d'un écran à un autre s'effectue via la classe FXMLLoader, appliquant le code à une Scene affichée dans le Stage.

```
FXMLLoader loader = new FXMLLoader(getClass().getResource(name: "/com/cardealer/vehicletable.fxml"));
Parent root = loader.load();
Stage stage =(Stage) button_Save.getScene().getWindow();
stage.setScene(new Scene(root, width: 1200, height: 900));
stage.setMaxWidth(value: 1200);
stage.setMaxHeight(value: 900);
stage.setTitle(value: "Vehicle Table");
stage.show();
```

4. Étape 4 : Connexion à la Base de Données

Pour relier l'application JavaFX à MySQL, une classe dédiée DBConnection a été implémentée. Elle utilise le driver JDBC et la méthode DriverManager.getConnection() pour établir un canal de communication sécurisé. Cette approche centralisée permet d'appeler DBConnection.getConnection() depuis n'importe quel contrôleur.

```

package com.cardealer.DAO;  Rename this package name to match the regular expression '^[a-z_]+(\.[a-z_][a-z0-9_]*)*$

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;

public class DBConnection {  Add a private constructor to hide the implicit public one.

    public static Connection getConnection() throws SQLException {

        try {
            // Charge le pilote JDBC MySQL
            Class.forName(className: "com.mysql.cj.jdbc.Driver");
        } catch (ClassNotFoundException e) {
            e.printStackTrace();  Print Stack Trace
            throw new SQLException(reason: "MySQL JDBC driver not found", e);
        }

        // Retourne la connexion établie à la base de données
        return DriverManager.getConnection(url: "jdbc:mysql://localhost:3306/carDealer", user: "root", password: "");
    }
}

```

5. Étape 5 : Implémentation du CRUD (Create, Read, Update, Delete)

C'est le cœur de l'application. La gestion des véhicules illustre parfaitement les fonctionnalités de persistance des données.

1. Ajout d'un véhicule (CREATE)

- **Upload d'image** : Utilisation de FileChooser pour sélectionner une image convertie ensuite en FileInputStream.

```

buttonuploadVehiclePicture.setOnAction(new EventHandler<ActionEvent>() {  This anonymous inner class creation can be turned into a lambda expression.
    @Override
    public void handle(ActionEvent event) {
        FileChooser fileChooser = new FileChooser();
        fileChooser.setTitle(value: "Upload Vehicle Picture");
        fileChooser.getExtensionFilters().addAll(new FileChooser.ExtensionFilter(description: "Image Files", ...extensions: "*.jpg", "*.png", "*.jpeg", "*.svg"));
        Stage stage = (Stage) imageviewaddnewvehicle.getScene().getWindow();

        File file = fileChooser.showOpenDialog(stage);
        if (file == null) {
            return;  Unnecessary return statement
        } else {
            try {
                path = file.getAbsolutePath();
                System.out.println(path);  Replace this use of System.out by a logger.
                Image image = new Image(new FileInputStream(path));
                imageviewaddnewvehicle.setImage(image);
            } catch (FileNotFoundException e) {
                throw new RuntimeException(e);  Replace generic exceptions with specific library exceptions or a custom exception.
            }
        }
    }
});

```

- **Sécurisation des saisies** : Vérification des champs vides et formatage rigoureux des données numériques.


```

if(input_vehicle_name.isEmpty() || input_manufacturer.isEmpty() ||
    input_construction_year.isEmpty() || km_stood.isEmpty() ||
    condition == null || currency == null || pieces.isEmpty() ||
    txt_price.getText().isEmpty() || path == null) {

    Alert alert = new Alert(Alert.AlertType.ERROR);
    alert.setTitle(title: "Input error");
    alert.setContentText(contentText: "Input or select fields cannot be empty");
    alert.show();

    return;    Unnecessary return statement
} else {

```

- **Requête Paramétrée** : Utilisation de PreparedStatement pour exécuter INSERT INTO, prévenant les injections SQL.

```

String sql = "INSERT INTO vehicle (vehicle_name, manufacturer, construction_year, km_stood, condition, number_of_pieces, price, currency, image) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)";
PreparedStatement pre = con.prepareStatement(sql);    Use try-with-resources or close this "PreparedStatement" in a "finally" clause.

pre.setString(parameterIndex: 1, input_vehicle_name);
pre.setString(parameterIndex: 2, input_manufacturer);
pre.setString(parameterIndex: 3, input_construction_year);
pre.setDouble(parameterIndex: 4, input_kmStood);
pre.setString(parameterIndex: 5, condition);
pre.setInt(parameterIndex: 6, input_pieces);
pre.setDouble(parameterIndex: 7, input_price);
pre.setString(parameterIndex: 8, currency);
pre.setBlob(parameterIndex: 9, fileInputStream);

pre.executeUpdate();

FXMLLoader loader = new FXMLLoader(getClass().getResource(name: "/com/cardealer/vehicletable.fxml"));
Parent root = loader.load();
Stage stage = (Stage) button_Save.getScene().getWindow();
stage.setScene(new Scene(root, width: 1200, height: 900));
stage.setMaxWidth(value: 1200);
stage.setMaxHeight(value: 900);
stage.setTitle(value: "Vehicle Table");
stage.show();

catch (SQLException | IOException e) {
    throw new RuntimeException(e);    Replace generic exceptions with specific library exceptions or a custom exception.
}
catch (NumberFormatException e) {

    Alert alert = new Alert(Alert.AlertType.ERROR);
    alert.setTitle(title: "Format error");
    alert.setContentText(contentText: "Please enter valid numbers for Price, Km and Pieces.");
    alert.show();
}

```

2. Affichage des véhicules (READ)

- Extraction via une requête SELECT.

```

Connection con = DBConnection.getConnection();
PreparedStatement pre = con.prepareStatement(sql: "SELECT * FROM vehicle WHERE id = ?");
pre.setInt(parameterIndex: 1, id_number);
ResultSet rs = pre.executeQuery();

```

- Stockage des résultats (ResultSet) dans une ObservableList.

```

while (rs.next()){
    observableList.add(new ModelTableVehicle(
        rs.getString(columnLabel: "id"),
        rs.getString(columnLabel: "name"),
        rs.getString(columnLabel: "manufacturer"),
        rs.getString(columnLabel: "construction_year"),
        rs.getString(columnLabel: "km_stood"),
        rs.getString(columnLabel: "condition"),
        rs.getString(columnLabel: "number_of_pieces"),
        rs.getString(columnLabel: "price"),
        rs.getString(columnLabel: "currency"),
        availability: "Yes" // Remplacement temporaire s
    ));
}

System.out.println(observableList);

```

- Liaison à un TableView grâce aux PropertyValueFactory.

```

tablecolumn_id.setCellValueFactory(new PropertyValueFactory<>(property: "vehicle_id"));
tablecolumn_vehicle_name.setCellValueFactory(new PropertyValueFactory<>(property: "vehicle_name"));
tablecolumn_manufacturer.setCellValueFactory(new PropertyValueFactory<>(property: "manufacturer"));
tablecolumn_construction_year.setCellValueFactory(new PropertyValueFactory<>(property: "construction_year"));
tablecolumn_km_stood.setCellValueFactory(new PropertyValueFactory<>(property: "km_stood"));
tablecolumn_vehicle_condition.setCellValueFactory(new PropertyValueFactory<>(property: "vehicle_condition"));
tablecolumn_pieces.setCellValueFactory(new PropertyValueFactory<>(property: "pieces"));
tablecolumn_price.setCellValueFactory(new PropertyValueFactory<>(property: "price"));
tablecolumn_currency.setCellValueFactory(new PropertyValueFactory<>(property: "currency"));
tablecolumn_availability.setCellValueFactory(new PropertyValueFactory<>(property: "availability"));

```

3. Transfert de données et Modification (UPDATE)

- **Transfert du contexte** : Les informations sont stockées dans des variables public static récupérées dans initialize().

```

button_upload_picture.setOnAction(new EventHandler<ActionEvent>() {
    @Override
    public void handle(ActionEvent event) {

```

- **Mise à jour** : Exécution d'une requête UPDATE vehicle SET... WHERE vehicle_id = ?.

```

Connection con = DBConnection.getConnection();

FileInputStream fileInputStream = new FileInputStream(path);

PreparedStatement pre = con.prepareStatement(sql: "UPDATE vehicle SET picture = ? WHERE vehicle_id = ?");

```

4. Suppression (DELETE)

- Action protégée par une boîte de dialogue de confirmation (Alert de type CONFIRMATION).

```

public void handle(ActionEvent event) {

    Alert alert = new Alert(Alert.AlertType.CONFIRMATION);

    alert.setTitle(title: "Warning");

    alert.setContentText(contentText: "Are you sure ? Do you really want to delete this vehicle ?");

    Optional<ButtonType> result = alert.showAndWait();

    if(result.get()==ButtonType.CANCEL){    Call "result.isPresent()" or "!result.isEmpty()" before acce

        return;    Unnecessary return statement
    }
}

```

- Exécution de DELETE FROM vehicle WHERE vehicle_id = ? suivie d'un rafraîchissement.

```

try {

    Connection con = DBConnection.getConnection();

    PreparedStatement pre = con.prepareStatement(sql: "DELETE FROM vehicle WHERE vehicle_id = ?");    Use
    // Ajoute cette ligne pour indiquer quel véhicule supprimer
    pre.setInt(parameterIndex: 1, VehicleTableController.id_number);
    pre.executeUpdate();

    pre.executeUpdate();

    FXMLLoader loader = new FXMLLoader(getClass().getResource(name: "/com/cardealer/vehicletable.fxml"));
    Parent root = loader.load();

    Stage stage = (Stage) button_delete.getScene().getWindow();

    stage.setScene(new Scene(root, width: 1200,height: 900));

    stage.setMaxWidth(value: 1200);

    stage.setMaxHeight(value: 900);

    stage.setTitle(value: "vehicle table");

    stage.show();
}

```

Conclusion et Suite du Projet

Ce TP a permis d'acquérir une compréhension solide de la création d'applications de bureau interactives. La gestion des exceptions et la manipulation des flux de données (images Blob) ont été des étapes formatrices. La prochaine phase consistera à adapter cette même logique pour les entités "Vendeurs" et "Clients" puisque les seuls changements sont les noms à remplacer dans les requêtes SQL.